24DCS120

CEUC102-PROGRAMMING IN C++

EXTERNAL PRACTICAL EXAM

AIM - The central library of a university plans to digitize its book management operations. the library maintains a wide collection of books , broadly categorized into types such as fiction and non fiction. each book is uniquely identified by its book id, and contains essential information such as title and author. additionally, books have specific classification details depending on their type:

1) a fiction book might belong to a category like mystery or fantasy

2) a non-fiction book might fall under a category like science or history.

Test case 1

```
24DCS120
RUTVI SHAH
BATCH-B

Library Management System
1. Add New Book
2. Display All Books
3. Calculate Fine
4. Exit
Enter your choice: 1
Enter Book ID: 5
Enter Title: xyz
Enter Author: abc
Enter Category (Fiction/Mystery or Non-Fiction/History): Fiction
Book added successfully.
24DCS120
RUTVI SHAH
BATCH-B

Library Management System
1. Add New Book
2. Display All Books
3. Calculate Fine
4. Exit
Enter your choice: 1
Enter Book ID: 6
Enter Title: pqr
Enter Author: hgf
Enter Category (Fiction/Mystery or Non-Fiction/History): Fiction
Book added successfully.
24DCS120
RUTVI SHAH
BATCH-B

Library Management System
1. Add New Book
2. Display All Books
3. Calculate Fine
4. Exit
Enter your choice: 1
Enter Book ID: 7
Enter Title: ff
Enter Author: jj
Enter Category (Fiction/Mystery or Non-Fiction/History): Non-Fiction
Book added successfully.
24DCS120
RUTVI SHAH
```

24DCS120

Test case 2

```
24DCS120
RUTVI SHAH
BATCH-B

Library Management System
1. Add New Book
2. Display All Books
3. Calculate Fine
4. Exit
Enter your choice: 1
Enter Book ID: 5
Enter Title: sdf
Enter Author: ghj
Enter Category (Fiction/Mystery or Non-Fiction/History): Fiction
Error: Duplicate entry. A book with this ID already exists.
24DCS120
RUTVI SHAH
BATCH-B

Library Management System
1. Add New Book
2. Display All Books
3. Calculate Fine
4. Exit
Enter your choice:
```

Test case 3

```
24DCS120
RUTVI SHAH
BATCH-B

Library Management System
1. Add New Book
2. Display All Books
3. Calculate Fine
4. Exit
Enter your choice: 1
Enter Book ID: 66
Enter Title: 1984
Enter Author: George Orwell
Enter Category (Fiction/Mystery or Non-Fiction/History): Mystery
Error! File not found
Book added successfully.
24DCS120
RUTVI SHAH
BATCH-B

Library Management System
1. Add New Book
2. Display All Books
3. Calculate Fine
4. Exit
Enter your choice:
```

24DCS120

Test case 4

```
24DCS120
RUTVI SHAH
BATCH-B

Library Management System
1. Add New Book
2. Display All Books
3. Calculate Fine
4. Exit
Enter your choice: 1
Enter Book ID: 88
Enter Title: 1984
Enter Author: George Orwell
Enter Category (Fiction/Mystery or Non-Fiction/History): History
Error! No records found
Book added successfully.
24DCS120
RUTVI SHAH
BATCH-B

Library Management System
1. Add New Book
2. Display All Books
3. Calculate Fine
4. Exit
Enter your choice:
```

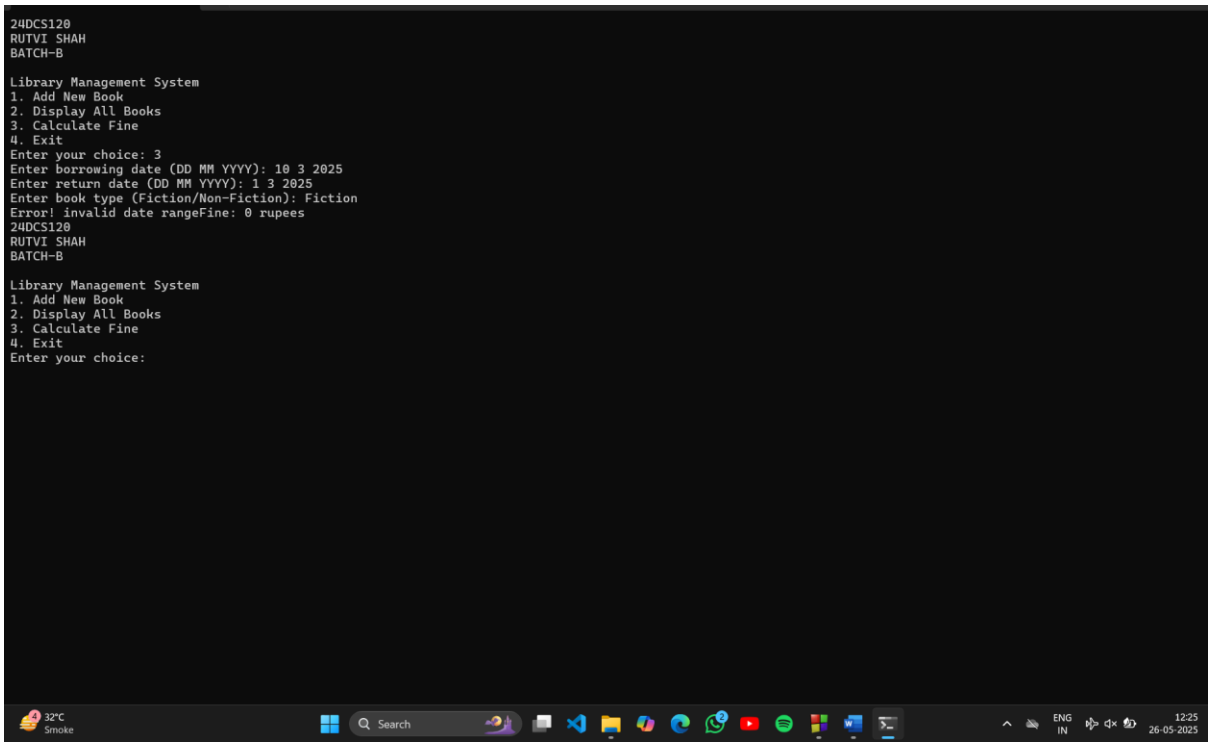
Test case 5

```
24DCS120
RUTVI SHAH
BATCH-B

Library Management System
1. Add New Book
2. Display All Books
3. Calculate Fine
4. Exit
Enter your choice: 3
Enter borrowing date (DD MM YYYY): 1 1 2025
Enter return date (DD MM YYYY): 17 1 2025
Enter book type (Fiction/Non-Fiction): Fiction
Fine: 140 rupees
24DCS120
RUTVI SHAH
BATCH-B

Library Management System
1. Add New Book
2. Display All Books
3. Calculate Fine
4. Exit
Enter your choice:
```

Test Case 6



CODE

```
#include <iostream>

#include <fstream>

#include <sstream>

#include <vector>

#include <stdexcept>


using namespace std;
```

```
class Book
{
protected:
   int bookID;
   string title;
   string author;
   string category;

public:
   Book(int id, string t, string a, string c)
{
   bookID=id;
   title=t;
   author=a;
   category=c;
}
   virtual void displayDetails() const
{
      cout << "Book ID: " << bookID << "\nTitle: " << title << "\nAuthor: " <<
author << "\nCategory: " << category << endl;
   }

   int getBookID() const { return bookID; }
   string getDetails() const {
      return to_string(bookID) + "," + title + "," + author + "," + category;
   }
```

```
};

class FictionBook : public Book {
public:
    FictionBook(int id, string t, string a, string c) : Book(id, t, a, c) {}
};


class NonFictionBook : public Book {
public:
    NonFictionBook(int id, string t, string a, string c) : Book(id, t, a, c) {}
};


bool bookExists(int id) {
    ifstream file("books.txt");
    string line;
    while (getline(file, line)) {
        istringstream iss(line);
        int existingID;
        string temp;
        getline(iss, temp, ',');
        existingID = stoi(temp);
        if (existingID == id) return true;
    }
    return false;
```

```
}


void addBook() {
    int id;
    string title, author, category;

    cout << "Enter Book ID: ";
    cin >> id;
    cin.ignore();
    cout << "Enter Title: ";
    getline(cin, title);
    cout << "Enter Author: ";
    getline(cin, author);
    cout << "Enter Category (Fiction/Mystery or Non-Fiction/History): ";
    getline(cin, category);


    if (bookExists(id)) {
        throw runtime_error("Error: Duplicate entry. A book with this ID already
exists.");
    }

    ofstream file(".txt", ios::app);
    if (!file) throw runtime_error("Error: Cannot open file.");

    Book* book;
```

```
    if (category == "Fiction" || category == "Mystery") {
        book = new FictionBook(id, title, author, category);
    } else {
        book = new NonFictionBook(id, title, author, category);
    }

    file << book->getDetails() << endl;
    file.close();
    delete book;

    cout << "Book added successfully.\n";
}

void displayBooks() {
    ifstream file("books.txt");
    if (!file) throw runtime_error("Error: Cannot open file.");
    if (file.peek() == EOF) throw runtime_error("No records found.");

    string line;
    while (getline(file, line)) {
        cout << line << endl;
    }
    file.close();
}

void calculateFine() {
```

```
    int borrowDay, borrowMonth, borrowYear, returnDay, returnMonth,
returnYear;
    string bookType;

    cout << "Enter borrowing date (DD MM YYYY): ";
    cin >> borrowDay >> borrowMonth >> borrowYear;
    cout << "Enter return date (DD MM YYYY): ";
    cin >> returnDay >> returnMonth >> returnYear;
    cout << "Enter book type (Fiction/Non-Fiction): ";
    cin >> bookType;
    cout<<"Error! invalid date range";

    int borrowDays = borrowYear * 360 + borrowMonth * 30 + borrowDay;
    int returnDays = returnYear * 360 + returnMonth * 30 + returnDay;
    int overdueDays = returnDays - borrowDays;

    int fine = 0;
    if (overdueDays > 7 && overdueDays <= 15) {
        fine = (bookType == "Fiction") ? 15 * (overdueDays - 7) : 10 * (overdueDays
- 7);
    } else if (overdueDays > 15) {
        fine = (bookType == "Fiction") ? (15 * 8) + (20 * (overdueDays - 15)) : (10 *
8) + (15 * (overdueDays - 15));
    }

    cout << "Fine: " << fine << " rupees\n";
}
```

```
int main() {
   int choice;
   while (true) {
         cout<<"24DCS120\n";
   cout<<"RUTVI SHAH\n";
   cout<<"BATCH-B\n";
      cout << "\nLibrary Management System\n";
      cout << "1. Add New Book\n2. Display All Books\n3. Calculate Fine\n4.
Exit\nEnter your choice: ";



      cin >> choice;


      try {
         switch (choice) {
            case 1: addBook(); break;
            case 2: displayBooks(); break;
            case 3: calculateFine(); break;
            case 4: cout << "Exiting...\n"; return 0;
            default: cout << "Invalid choice. Try again.\n";
         }
      } catch (const exception& e) {
         cout << e.what() << endl;
      }
   }}
```